

vigilant-engine — Threat Modeling Plan

STRIDE + OWASP Threat Dragon + ATT&CK, layered on top of the scanner/ingestion pipeline

Working plan — 2026-09-05 · reference alongside ARCHITECTURE.md and PLAN.md

Sequencing decision: the manual threat model (Phase 2) happens **before** the scanners are pointed at the finalized target (Phase 3) — not before the tool-agnostic ingestion plumbing, which already exists and doesn't care what target it's fed. Threat-model-first means the coverage table in Phase 4 reads as "I predicted these threats independently, then checked tooling coverage" rather than "I ran tools, then noticed gaps afterward" — same content, much stronger claim about judgment.

PHASE 0

STRIDE fundamentals CONCEPTS LEARNED

1. DFD notation understood — the four element types (external entity, process, data store, data flow), and how to read/draw one. CONCEPT LEARNED

Not the same as "Juice Shop's DFD is done" — that's a separate artifact, still pending, in Phase 2 below.

2. Trust boundaries CONCEPT LEARNED

What it is: a line drawn across the DFD, cutting through specific data-flow arrows, marking a point where the level of trust or control changes between what's on either side. Not every arrow crosses one — only the ones where you'd genuinely have to verify something (identity, permission) before letting data through, if you were being careful.

Common kinds, so you can recognize one when you see it:

- **Network boundary** — internet vs. your internal network
- **Process/machine boundary** — data leaving one process/container and entering another you don't fully control
- **Privilege boundary** — anonymous user vs. authenticated user, regular user vs. admin
- **Physical boundary** — on-prem vs. cloud, your infra vs. a third party's

How to find them, mechanically: go arrow by arrow and ask one question — "do both ends of this arrow trust each other equally, or does crossing it mean going from something I don't control to something I do (or from lower privilege to higher)?" If yes to the second, draw a boundary line across that arrow. If both ends are things you fully control at the same privilege level, no boundary — it's just internal plumbing.

Why STRIDE cares: threats cluster at boundaries, not in the interior. An attacker has to *cross* a boundary to do damage — the one point where they're forced to either hold valid credentials or exploit something that lets them skip needing them. Everything sitting entirely on one side of every boundary has a much smaller realistic threat surface than anything a boundary-crossing arrow touches.

Applied to the practice sketch: CI → POST /ingest (boundary — CI is outside your control), Process ↔ DB INSERT/UPDATE/SELECT (no boundary — your own API talking to its own DB, same trust level), Process → GET /findings, /stats → Dashboard (boundary — a separate, externally-driven client). One boundary line covers both crossings, since they're the same boundary conceptually: "anything outside vigilant-engine's own process."

3. Element-type → applicable-STRIDE-category mapping CONCEPT LEARNED

Element	S	T	R	I	D	E
External entity	✓				✓	

Process	✓	✓	✓	✓	✓	✓
Data flow	✓			✓	✓	
Data store	✓	if log	✓	✓	✓	

Why each cell is what it is (worth understanding, not memorizing):

- **Spoofing** only applies to things that have or claim an identity — a user, a service, a process. A data flow or a data store isn't "someone," so it can't be impersonated. That's why only External entity and Process get an S.
- **Repudiation** only applies to things that take an action that could later be denied — a user denying they did something, a process denying it handled a request. A data store gets it only when it's specifically a log — a log's entire job is being the evidence that defeats repudiation; if that log can be silently altered or deleted, repudiation becomes possible again. `findings.db` isn't a log in that sense (it's primary data, not an audit trail), so no R for it.
- **Tampering** and **Information Disclosure** apply to anything that holds or carries data — flows (data in transit can be intercepted/modified) and stores (data at rest can be read/altered). Doesn't apply to an external entity itself — you don't "tamper with a user," you tamper with what they send, which is a flow.
- **Denial of Service** applies to anything that must stay available — flows can be flooded, stores can be exhausted/locked, processes can be crashed. External entities are outside your system, so keeping them "available" isn't your problem.
- **Elevation of Privilege** only applies to a Process — it's the only element type that actually executes with a privilege level. A data store or a flow has no privilege to escalate into.

Applied to the practice sketch, row by row:

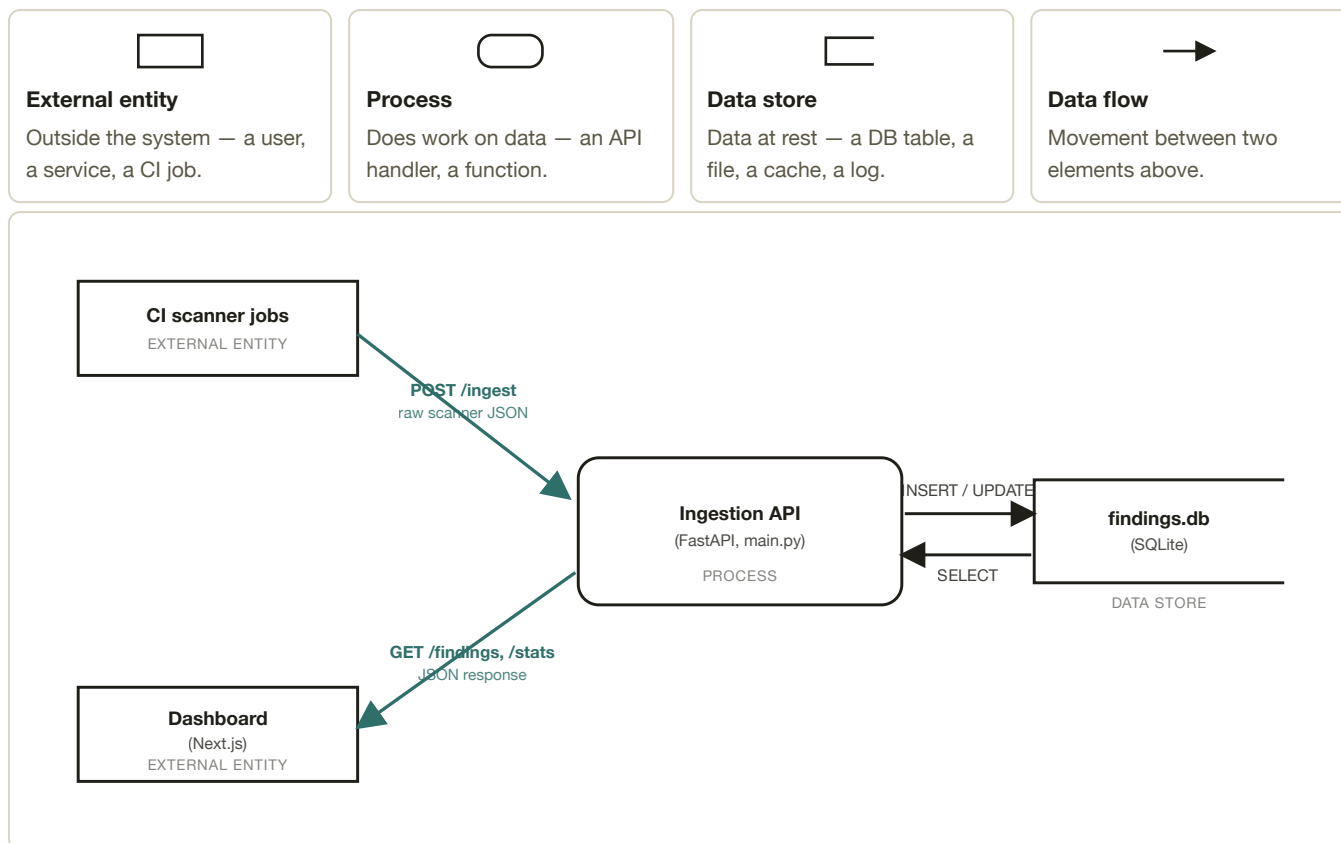
- `CI_scanner_jobs` (external entity) → Spoofing, Repudiation
- `Dashboard` (external entity) → Spoofing, Repudiation
- `Ingestion_API` (process) → all six
- `findings.db` (data store, not a log) → Tampering, Information Disclosure, DoS
- Each data flow (`POST /ingest`, `INSERT/UPDATE`, `SELECT`, `GET`) → Tampering, Information Disclosure, DoS

What falls out for free: the Process is the only element carrying all six threat categories, and it's also the only element sitting on both sides of the trust boundary — exactly why it's the highest-value thing to scrutinize. The table isn't just bookkeeping; it tells you where to spend your attention.

All three items are concept-level — the systematic walk itself (applying this table row-by-row to real elements) is the work of Phase 2, not yet done.

Practice sketch — NOT Juice Shop, just for learning the notation

This diagram is vigilant-engine's own ingestion service — familiar code, used only to rehearse spotting the four element types before doing this for real. It does not count toward Phase 2. Juice Shop's DFDs (login/auth, checkout, order/profile access) still need to be drawn from scratch in Phase 2, step 2.



Teal marks the two flows crossing from an external entity into the ingestion API — the only two places something outside your control reaches in. Neither currently checks who's on the other end; that gap is what trust-boundary analysis (Phase 0, step 2) is built to surface. Reflects *ingestion/app/main.py* and *ingestion/app/db.py* as of this session.

PHASE 1

Lock and stand up the target DONE

- Juice Shop wired in as a git submodule (`demo-target/` → `github.com/juice-shop/juice-shop`), pinned to `v20.2.0`.
- Verified running locally via `npm start` (dependencies were already installed in the submodule) — reachable and working.
- Docker was checked and isn't installed on this machine yet — not a blocker for now; noted as a Phase 3 dependency, since the ZAP DAST job and CI will want a pinned `bkimminich/juice-shop:v20.2.0` container so the scan target is reproducible, rather than however it happens to run locally.

Not yet done, and not a Phase 1 blocker: the whole repo (including this submodule reference) is still staged, not committed — there are no commits on this branch yet. That's a separate decision (when to make the initial commit), not something this phase depends on.

PHASE 2

Manual threat model (the part that proves judgment) NOT STARTED

Target: Juice Shop, not vigilant-engine. Nothing in this phase exists yet — the practice sketch above doesn't count.

The approach taken for Phase 2 — how to actually assess the data flow

Not pure static code reading — the standard AppSec method blends **black-box traffic observation** and **white-box source reading**, with a debugger as an occasional deep-dive tool, not the default. In priority order, per flow:

1. **Watch the real traffic first (black-box).** Before touching source, perform the action live in the running app and watch what actually crosses the wire — code can have dead paths or behavior that differs from what a quick read suggests, so this is the most reliable way to know the real data flow.
 - **Browser DevTools** → **Network tab** — zero install, works immediately against the running Juice Shop instance. Inspect the exact URL, method, request body, response body, and any headers/cookies set.
 - **An intercepting proxy** — Burp Suite (Community, free) or OWASP ZAP (already in the scanner list) sit between browser and server and let you see, and even modify, every request live. Worth learning regardless of this project — one of the most standard AppSec tools there is. Installed via `brew install --cask burp-suite` (Community Edition, free) — no VM/Kali needed, it's a regular desktop app. Setup: launch → Temporary project → Use Burp defaults → Proxy tab → Intercept → **Open Browser** (Burp's own pre-configured Chromium, certificate already trusted) → browse the app in that window; every request lands in **Proxy** → **HTTP History**. Core workflow: right-click a request → **Send to Repeater** to edit and resend it repeatedly (replaces retyping curl commands); **Comparer** to diff two responses side by side; **Target** → **Site map** to see every endpoint touched this session.

This step answers the **data flow arrows** precisely: what's actually in the request (just email + password, or more?), what's in the response (a JWT? where — body or cookie? does it leak more fields than needed?).
2. **Read the source second (white-box)**, now that the endpoint is known. Traffic capture can't show server-side logic — only inputs and outputs. Grep for the exact route (e.g. `POST /rest/user/login`) in `demo-target/routes/`, follow it into the handler, read it step by step: DB query, password comparison, token issuance. This fills in the **Process** internals.
3. **For the DB layer, don't guess from model code — turn on query logging.** Juice Shop uses Sequelize, which has a config option to print every SQL statement it runs to the console. Flip it on, trigger the action again, and see the literal `SELECT/INSERT` executed — more reliable than reading model definitions and inferring the query.
4. **A debugger is the last resort, not the first move.** Attaching Node's inspector (`node --inspect`) with a breakpoint in the handler is useful when the first three steps leave something ambiguous (e.g. which of two code paths actually executes) — but too slow to use as the default method across a whole flow.

Why this order, not source-reading alone: black-box traffic tells you ground truth about what's transmitted; white-box reading tells you what happens inside the box; query logging confirms the data store interaction directly; the debugger is for resolving ambiguity, not for first-pass mapping. Using only source reading risks modeling code paths that aren't actually reachable, or missing behavior that only shows up at runtime.

Since the target is a REST API, cross-check coverage afterward against the OWASP API Security Top 10 (2023) — as a coverage check, not the thing driving exploration. Each row is what to actually look for in the Network tab, not just the category name:

#	Category	What to look for in Network tab
API1	Broken Object Level Authorization	Any URL/param holding an object ID — numeric, sequential, or guessable. Swap it while authenticated as yourself.
API2	Broken Authentication	Does the token expire (exp claim)? Any rate-limit on repeated failed logins (watch for 429)? Predictable reset tokens?
API3	Broken Object Property Level Authorization	Response fields the UI never displays (over-exposure); on PUT/PATCH, try adding a field you didn't send (e.g. <code>"role": "admin"</code>) — accepted (mass assignment)?
API4	Unrestricted Resource Consumption	Pagination/quantity params — send an extreme value, see if the server just complies with no cap.
API5	Broken Function Level Authorization	Admin-only-looking endpoints called with a regular user's token — 200 instead of 401/403?

API6	Unrestricted Access to Sensitive Business Flows	Business actions with no throttle — repeat a "submit review"/"apply coupon" POST rapidly; anything ever blocks it?
API7	Server-Side Request Forgery	Any request/field accepting a URL as input (image-via-URL, a webhook/import feature) — flag for later testing.
API8	Security Misconfiguration	Response headers: missing CSP/X-Content-Type-Options, exposed Server/X-Powered-By, permissive CORS, verbose errors.
API9	Improper Inventory Management	Undocumented/old API versions — check for an exposed Swagger/OpenAPI doc revealing endpoints the UI never calls.
API10	Unsafe Consumption of APIs	About the server's trust in third-party APIs it calls — not visible in your own browser traffic; needs source review.

Source: owasp.org/API-Security/editions/2023. Findings from actually running this sweep against Juice Shop live in `THREAT_MODEL.md`, Flow 4.

1. Pick 2–3 bounded flows, not the whole app: login/auth, basket/checkout, order/profile access.
2. Draw a DFD for each flow (browser, REST API, JWT check, DB, admin logic) — **this is Juice Shop's DFD, still to be drawn.**
3. Mark trust boundaries on each.
4. Walk STRIDE per element using the Phase 0 type-mapping; brainstorm a concrete threat per applicable (element, category) pair.
5. For each threat, note: security property violated, plausible impact, and whether it matches a known Juice Shop challenge.
6. Transcribe the finished model into OWASP Threat Dragon — shareable diagram + JSON export, checked into the repo.

Phase 2 stock-take — 2026-09-07 checkpoint

Honest status against the six steps above, not just "done/not done":

Step	Status
1. Pick 2–3 bounded flows	Loosely done — Login/Auth, Basket, whoami, plus an emergent "Flow 4" (functional-authorization sweep) that wasn't planned but produced the best findings
2. Draw a DFD per flow	Not done. Only the practice sketch (vigilant-engine's own ingestion service) exists — no Juice Shop DFD yet
3. Mark trust boundaries	Not done — no DFD exists yet to mark them on
4. Walk STRIDE per element	Done <i>informally</i> — each finding got a STRIDE category after the fact, not via the structured element × category walk
5. Note property/impact/challenge match	Done well — every finding in <code>THREAT_MODEL.md</code> has this
6. Transcribe into Threat Dragon	Not done

What actually happened instead: black-box (DevTools/Burp) and white-box (source review) exploratory testing, guided by the OWASP API Security Top 10 rather than a formal DFD-first walk. Result: **9 confirmed findings** (JWT over-exposure; two flavors of basket BOLA — a missing check and a bypassed one; mass user-table disclosure; unauthorized product-price tampering; no login rate limiting; non-expiring JWTs; CORS/CSP gaps; exposed API schema), 2 open candidates, and 3 OWASP categories (API4, API6, API7) not yet touched. Full detail in `THREAT_MODEL.md`.

Roadblocks and conceptual corrections made along the way (kept here so this checkpoint is reviewable, not just a status line): confusing a "bounded flow" (an analyst's scoping choice — pick a tractable slice of the app) with a "trust boundary" (a property of the system — where trust level actually changes) because they share a word; assuming trust boundaries only separate privilege *levels* (customer vs. admin) until the basket BOLA showed they also separate *identities* at the same level (your data vs. a peer's); briefly misattributing the basket-tampering bug to the `clarinet` parsing library, when the library behaved correctly and the bug was Juice Shop's own inconsistent use of its output. None of these blocked progress — each got corrected in-session — but worth keeping visible here rather than only in the polished final writeup, since "what went sideways and got fixed" is itself useful interview material.

Not wasted effort: steps 2/3/6 remain, but formalizing them now means populating a DFD and Threat Dragon file with *real, already-confirmed* findings rather than hypothetical ones — arguably a better order than doing it the other way round.

PHASE 3

Set up and run the scanners

1. Semgrep, npm audit, Snyk, gitleaks against the source, in CI.
2. OWASP ZAP baseline scan against the running instance.
3. Trivy against the built `ingestion/` and `dashboard/` images.
4. Feed every tool's raw output into the already-built ingestion API.

PHASE 4

Coverage table (the punchline artifact)

1. For every threat identified in Phase 2, check: did any Phase 3 scanner actually surface it?
2. Record: tool, yes/no, and why (or why not) — this is the evidence that automated tooling alone wouldn't have found it.

PHASE 5

ATT&CK mapping — only where it's honest

1. For threats resulting in a foothold or privilege gain (e.g. admin bypass), map to the relevant ATT&CK tactic/technique.
2. For pure app-logic threats with no real ATT&CK analog, write "N/A" explicitly rather than forcing a mapping.

PHASE 6

Document and surface it

1. Write it up as `THREAT_MODEL.md` (DFDs, STRIDE threat list, Threat Dragon export, coverage table, ATT&CK annotations).
2. Link it prominently from the main README — this is the artifact doing the CV/interview work, so it shouldn't be buried.